

SECTION 27 — TESTING STRATEGY AND QA FUNDAMENTALS (Refined)

Topics Covered

- 27.1-27.18 (18 headers)

First pass — 2 parallel batch subagents

27.1 Test Pyramid — Refined

Theory: Mike Cohn's Test Pyramid groups tests by granularity and prescribes volume: many fast unit tests at base, fewer service/integration tests in middle, very few UI/E2E tests at top. Rationale: speed (seconds vs minutes), cost (cheap to write/run/debug), maintainability (precise failure isolation). Lower layers give rapid feedback; upper layers give confidence but slowly. Anti-pattern: Ice-Cream Cone (dominant manual/E2E — slow, brittle, expensive). SDET use: enforce ratio in CI, gate merges on unit coverage. Enterprise Relevance: Pyramid violations are visible in pipeline metrics: E2E-heavy suites take 2h and flake 15%; pyramid-conformant suites take 15min at 1% flake. The pyramid justifies pushing tests down (API over UI) with cost data.

```
Unit (70%) > Service/API (20%) > UI/E2E (10%); run base on every commit.  
Cone (anti): Manual 50% + E2E 40% + unit 10% = slow, flaky, unmaintainable.
```

High-Stakes Scenario: 80% E2E suite, 2h runtime, 20% flake — releases blocked weekly. Triage: audit suite by layer (count + runtime + flake each), migrate stable E2E assertions down to API (same coverage, 10x faster), reserve UI for critical journeys only, gate new UI tests (require API-impossibility justification). Anti-Patterns: Ice-cream cone; "automate everything via UI"; unit coverage gate without layer ratio (100% unit + 500 E2E still slow); testing library code (no value). Qs: Draw the pyramid with ratios? What proves a suite violates it?

27.2 Test Automation Pyramid — Refined

Theory: The Automation Pyramid operationalizes Cohn's pyramid for CI/CD: automate bottom-up, keep tests deterministic, isolated, and fast. Unit: JUnit/pytest with mocks (ms each). Service/API: RestAssured/Supertest with WireMock/Pact (100s of ms). UI: Selenium/Playwright, only critical journeys (seconds each). Add contract and component layers for microservices. SDET owns pipeline stages, parallelization, quarantine for flaky tests — and avoids duplicated coverage across layers (same assertion in unit + API + UI = triple maintenance). Enterprise Relevance: Pipeline stage design follows the pyramid: unit (2min) → api+contract (8min) → e2e-smoke (10min) → nightly full. Each layer has its own gate, runtime budget, and flake SLA.

```
Pipeline: unit(2min, gate:merge) -> api+contract(8min, gate:promote) -> e2e-  
smoke(10min, gate:staging) -> nightly full (informational)
```

High-Stakes Scenario: Same login assertion in unit + API + 5 UI tests — login API change breaks 7 tests, triage takes a day. Triage: one layer per fact (login logic → unit/API; login journey → 1 UI smoke), dedup audit quarterly, coverage-by-layer map. Anti-Patterns: Duplicated coverage across layers; UI

testing API logic (slow); no contract layer for microservices (integration gaps); E2E for unit-testable branches. Qs: What lives at each automation layer? How do you detect duplicated coverage?

27.3 Shift Left — Refined

Theory: Shift Left moves testing earlier: requirements review, static analysis, TDD, BDD, unit + API tests on PR, SAST/lint in IDE/pre-commit. Goal: defect prevention over late detection — cost-of-fix rises exponentially rightward (1x requirements → 10x code → 100x production, IBM/NIST). Deloitte/SDET context: participate in refinement, define acceptance criteria, add testability hooks (data-testid , API contracts), consumer-driven contracts. Metrics: escaped defects, PR rejection rate, time-to-feedback under 10 minutes. Enterprise Relevance: Shift-left maturity is what separates Senior SDET (prevents bugs via AC + hooks + gates) from test executors (finds bugs after merge). Every refinement without SDET input is untestable requirements shipped downstream.

```
Pre-commit: eslint + tsc + jest --changed (30s)
PR: SAST + API contract + unit coverage gate (8min)
```

High-Stakes Scenario: Untestable requirement ("fast checkout") ships; testing discovers 14 gaps post-coding (Sec 33.15 case) — 30% rework. Triage: example-mapping workshops (concrete examples before code), testability checklist in DoD (hooks, contracts, observability), AC with measurable criteria. Anti-Patterns: QA excluded from refinement; untestable AC accepted; SAST optional (never run); testing starting after code freeze. Qs: What shift-left artifacts do you produce in refinement? How do you measure shift-left maturity?

27.4 Shift Right — Awareness — Refined

Theory: Shift Right extends testing into production: observability (metrics/logs/traces), feature flags (kill-switch + gradual rollout), canary/blue-green deployments, A/B tests, chaos experiments, synthetic monitoring (probes hitting prod journeys), log/RUM analysis. SDET builds post-deploy verification, rollback triggers, production smoke probes, data validation, SLA/SLO alerts. Awareness means accepting unknown unknowns — testing in prod safely with blast-radius control (1% traffic, auto-rollback) and quick mitigation, feeding findings back left (gaps → new pre-prod tests). Enterprise Relevance: Pre-prod can never match prod (data volume, traffic mix, third-party behavior). Shift-right practices (canary analysis, prod probes, chaos) catch what staging cannot. Deloitte SRE-aligned QA owns post-deploy verification, not just pre-deploy gates.

```
Canary 5% → check error-rate/burn-rate (10min) → promote 25% → 100%; flag-off on SLO breach (auto).
Prod probes: synthetic login+checkout every 5min; alert on failure (faster than user reports).
```

High-Stakes Scenario: Staging-green release breaks prod (prod-only data shape: 10M-row table, missing index) — discovered by users in 2h. Triage: canary with automated analysis (would have caught in 10min at 5%), prod probes (5min detection vs 2h), RUM anomaly alerts, flag-off rollback in 3min. Anti-Patterns: Big-bang deploys (100% at once); no prod probes (users are monitors); flags without kill-switch testing; chaos only in theory; findings not fed back to pre-prod suite. Qs: Shift-left vs shift-right — what does each catch that the other cannot? How do you test in prod safely?

27.5 Risk-Based Testing — Refined

Theory: Risk-Based Testing prioritizes effort by $\text{Risk} = \text{Probability} \times \text{Impact}$, focusing on critical, complex, changed, or compliance-sensitive areas. Steps: identify risks (workshop + defect history + complexity), score likelihood $\times$ business impact, map to test types (E2E/API/unit/exploratory), allocate automation depth, define exit criteria (residual risk acceptance). Use risk matrix + requirement traceability. For Deloitte engagements: align with client risk appetite, audit needs, and limited timelines to maximize defect discovery where failure cost is highest. Enterprise Relevance: With finite time, risk order decides what gets tested. "Test everything" is not a strategy; risk-ranked backlog with coverage-per-risk is.

Risk Score: Payment-charge (5 likelihood $\times$ 5 impact = 25) = P1, automate E2E + contract + chaos.
Footer-link (1 $\times$ 2 = 2) = P3, manual/exploratory only.
Exit: all P1 risks have automated E2E + rollback tested; P2 API-covered; P3 sampled.

High-Stakes Scenario: 3-day deadline, 400 cases, 2 QAs — risk ranking delivers 100% P0 + key P1 in time, 12 critical bugs found, zero post-go-live incidents. Without ranking: random 30% coverage, P0s missed. Anti-Patterns: Alphabetical/priority-field testing (no risk basis); equal effort all areas; risk matrix created but ignored; no exit criteria (testing "until time runs out"). Qs: How do you score likelihood $\times$ impact? What exit criteria prove residual risk acceptable?

27.6 Regression Strategy — Refined

Theory: Regression ensures new changes do not break existing functionality via selective, progressive suites: smoke on deploy, P1/P2 automated regression on nightly/release, full suite pre-release. Maintain risk-tagged, modular suites with versioned test data and environment parity. Optimize with impact analysis (changed-service $\rightarrow$ affected tests), test selection, parallel sharding. Track flakiness, quarantine, auto-heal locators; require traceable pass evidence for enterprise sign-off. Enterprise Relevance: Regression is the release-confidence engine. Tiering (smoke $\rightarrow$ P1 $\rightarrow$ full) matches cost to risk at each gate; impact analysis prevents running 2000 tests for a copy change.

Tag: @regression @P1; mvn test -Dgroups=P1; Playwright --shard=1/4 --retries=2
Tiers: deploy-smoke (8min) $\rightarrow$ nightly P1/P2 (40min) $\rightarrow$ pre-release full (2h)

High-Stakes Scenario: Full 2h regression on every PR (developers wait, then bypass). Triage: impact-mapped selection (5% of suite per PR average), smoke gate (8min), nightly full, pre-release risk slice. PR feedback 2h $\rightarrow$ 9min, coverage preserved via tiering. Anti-Patterns: Untagged monolith (cannot slice); full suite per commit; no impact analysis; stale tests never pruned (suite grows 10%/quarter, value doesn't). Qs: How do you select tests per change? What tiers run at which gates?

27.7 Smoke Strategy — Refined

Theory: Smoke is broad, shallow build-acceptance: can we test further? Covers launch, login, core navigation, key API health, DB connectivity in 5-15 minutes. Runs on every build/deploy, blocks pipeline on failure. Keep stable, fast, environment-agnostic, no deep assertions. SDET maintains <20 cases owned as pipeline gate with clear notifications. Fail = reject build, no further regression waste. Enterprise Relevance: Smoke is the cheapest gate with highest leverage — 10min proving "systematically alive" before spending 2h on regression. Every Deloitte pipeline starts here; smoke red stops everything downstream automatically.

```
smoke.spec: goto / → login → expect dashboard → GET /health 200 → checkout ping → DB
select 1
Budget: <15min, <20 cases, 0 flakes tolerated (quarantine immediately if flaky)
```

High-Stakes Scenario: Broken build (login 500) runs full 2h regression — $2h \times 8$ shards wasted + misleading failures. Triage: smoke-first DAG (regression needs smoke-pass), auto-cancel downstream on smoke fail, smoke failure pages build author directly. Anti-Patterns: Deep assertions in smoke (belongs in regression); 50-case "smoke" (it's regression); flaky smoke tolerated (gate ignored within weeks); environment-specific smoke (fails on config, not product). Qs: Smoke vs sanity vs regression in one line each? What makes smoke trustworthy as a gate?

27.8 Sanity Strategy — Refined

Theory: Sanity is narrow, focused verification after a minor fix/change: does this function rationally work before deeper regression? Unscripted or lightly scripted — verify bug fix + adjacent logic. Unlike smoke's breadth, sanity depth-checks the touched module. Entry: after dev fix; exit: decides regression scope (pass → run related regression; fail → return). Document outcome quickly in Jira/Xray. Automate only if repeated; otherwise keep manual/exploratory for speed in sprint hotfixes. Enterprise Relevance: Sanity is the hotfix accelerator — 15min focused check vs 2h regression for a 1-line fix. It answers "safe to proceed?" fast, with regression following for confidence.

```
Sanity: defect-1234 coupon 10% applies → check total + boundary 0%/100% → PASS → run
pricing regression.
Timebox: 15-30min; output: PASS/FAIL + scope recommendation in ticket.
```

High-Stakes Scenario: Hotfix deployed without sanity (regression "takes too long") — fix broke adjacent coupon path, discovered in prod. Triage: mandatory sanity checklist per hotfix (fix + adjacent + smoke), timeboxed, ticketed; regression follows async. Anti-Patterns: Full regression for every typo fix (slow); no sanity (blind deploy); automating one-off sanity (waste); sanity as regression replacement (too narrow). Qs: Smoke vs sanity — breadth vs depth? When is manual sanity better than automated?

27.9 Integration Testing — Refined

Theory: Integration verifies interactions between modules/services: API-to-API, service-to-DB, queue, third-party. Scope: data serialization, contracts, auth, error handling, timeouts/retries. Approaches: narrow (with test doubles/WireMock locally, fast, deterministic) vs broad (with real test instances, realistic, slower). Complement with Pact contract tests. SDET spins dependencies via Docker/Testcontainers, isolates data, asserts cross-boundary state — not just status codes. Enterprise Relevance: Integration gaps cause the "works alone, breaks together" class — serialization mismatches, auth propagation failures, timeout cascades. Narrow for PR speed, broad nightly for realism.

```
WireMock.stubFor(get("/weather").willReturn(okJson("{\"temp: 21}")));
assertEquals(client.parse(), new Weather(21)); // contract + parsing, no network
// + Pact provider verification in CI for real-contract confidence
```

High-Stakes Scenario: Service A sends ISO dates, Service B expects epoch — unit tests green (mocked), integration red in staging (real serialization). Triage: contract tests (Pact) on every PR,

narrow integration with realistic doubles, broad nightly with real instances, serialization fuzz (dates, nulls, unicode). Anti-Patterns: Unit-only + E2E-only (missing middle); real third-parties in PR tests (slow/flaky); status-code-only asserts (miss state corruption); no timeout/retry testing. Qs: Narrow vs broad integration — when each? What does Pact add over integration tests?

27.10 System Testing — Refined

Theory: System Testing validates the complete integrated system against SRS/requirements as black-box in a production-like staging environment — after integration testing. Covers functional, UI, security, performance, recovery, compliance; driven by RTM (requirements traceability matrix) for coverage. Independent QA/SDET owns it. Entry: stable build, data + environment ready. Exit: critical defects fixed, exit criteria met. Deloitte angle: integration = interfaces between parts; system = whole behavior vs requirements; UAT = business acceptance by users. Enterprise Relevance: System testing is the contractual verification ("does it meet spec?") — RTM coverage proves every requirement tested. Regulated clients require RTM + evidence per requirement.

```
assert checkout(login→search→cart→pay→confirm) == success + DB + logs valid
RTM: REQ-101 → TC-201..205 (all pass) → requirement covered
```

High-Stakes Scenario: Requirement "refunds within 5 days" never tested (no RTM mapping) — violated in prod, penalty clause triggered. Triage: RTM enforced (every requirement ≥ 1 test, every test ≥ 1 requirement), unmapped requirements flagged pre-release, exit criteria include RTM 100%. Anti-Patterns: No RTM (coverage unknown); system testing on dev env (invalid); entry criteria ignored (testing unstable build wastes cycles); UAT confused with system (different owners/purposes). Qs: Integration vs system vs UAT in one table? What are entry/exit criteria?

27.11 End-to-End Testing — Refined

Theory: E2E validates entire business workflows across UI, APIs, microservices, DBs, third-party integrations from start to finish under real-user scenarios. Done after system testing in dedicated test environment, often by QA. Types: horizontal (user journey across features — Gmail login→compose→send→receive) vs vertical (layered data flow without UI — API→SQL). Catches dependency and data-integrity issues. Costly and slower — triage critical paths; automate stable smoke/regression E2E with Playwright/Cypress. Enterprise Relevance: E2E proves the business works, not just parts. But each E2E costs 100x a unit test (time + flake + maintenance) — reserve for critical journeys (login, checkout, payment), max ~10% of suite.

```
test("order E2E", async () => {
  await login(); await addToCart(); await checkout();
  expect(orderInDB).toBeTruthy(); expect(paymentCaptured).toBeTruthy();
});
```

High-Stakes Scenario: 500 E2E tests, 3h runtime, 15% flake — team stops trusting automation entirely. Triage: keep 20 critical-journey E2E (smoke + P0), migrate rest down (API contract + component), quarantine flakes, E2E budget (new E2E requires retiring one or API-impossibility proof). Anti-Patterns: E2E for everything (pyramid inversion); E2E with third-party live deps; no API seeding (30s UI setup per test); E2E asserting API logic (slow, misplaced). Qs: Horizontal vs vertical E2E? How do you justify a new E2E test?

27.12 Exploratory Testing — Refined

Theory: Exploratory Testing is simultaneous learning, test design, and execution without pre-scripted cases (coined by Cem Kaner). Ideal for new features, unstable requirements, or supplementing scripted regression to find subtle, UX, and edge bugs. Structured via test charters (mission + scope + timebox 60-90min), mind-maps, and Session-Based Test Management with notes/screenshots for reproducibility. Emphasizes tester autonomy and heuristics like SFDIPOT (Structure, Function, Data, Interfaces, Platform, Operations, Time). Deloitte tip: convert found bugs into automated regression scenarios; never use exploratory alone for compliance-auditable coverage. Enterprise Relevance: Exploratory finds what scripts miss (scripts only check known expectations). Timeboxed charters + SBTM debriefs make it manageable and reportable — "90min charter found 3 bugs, 2 converted to regression".

```
Charter: Explore checkout coupons | 60min | Mission: break pricing
Log: steps + screenshots + bugs (Jira links) + coverage notes
Debrief: what was tested, found, risks remaining
```

High-Stakes Scenario: New feature ships with scripted tests green but unusable UX flow (scripted checks fields, nobody tried the journey). Triage: exploratory charter per feature (before sign-off), pairing dev+QA for system tour, found bugs → regression conversion tracked. Anti-Patterns: Unstructured "click around" (unreportable); exploratory as sole coverage (unauditable); no charters/timeboxes (endless); findings not converted (same bugs recur). Qs: How do you make exploratory accountable (charter/SBTM)? When is exploratory better than scripted?

27.13 Defect Lifecycle — Refined

Theory: Defect Lifecycle tracks a bug from detection to closure: New → Open/Acknowledged (triaged) → Assigned → In Progress → Fixed/Resolved → Verified/Tested → Closed. Alternate states: Reopened (fix failed verification), Deferred (postponed with approval), Rejected/Not-a-Bug (working as designed + evidence), Duplicate. ISTQB chain: human Error → Fault/Defect (in code) → Failure (observed). Managed in Jira/Linear with severity, priority, steps, evidence per IEEE 1044 / ISO 29119-3. Deloitte focus: triage ownership (who decides in 24h), clear repro (steps + data + env), verification plus regression (fix + guard test) to prevent recurrence. Enterprise Relevance: Lifecycle discipline decides whether bugs get fixed or rot. SLA by severity (P1: fix in 24h + hotfix process; P3: backlog) + triage rotation + reopen-rate metric (high reopen = poor fixes or poor repro).

```
New→Open→Assigned→Fixed→Verified→Closed; fail→Reopened; postpone→Deferred
(approved); invalid→Rejected+evidence; dup→Duplicate(link)
Required fields: severity, priority, env, steps-to-repro, expected/actual,
attachments (trace/video), linked requirement/test
```

High-Stakes Scenario: "Fixed" bugs reopen 40% (dev can't repro, fixes wrong cause). Triage: repro quality gate (steps + data + trace mandatory), fix-verified-by-reporter rule, regression test attached to ticket before close, reopen-rate tracked per team. Anti-Patterns: Bugs closed without verification; missing repro steps ("it broke"); severity=priority confusion; deferred without approval/date; duplicates fixed twice. Qs: Walk the lifecycle states? Reopened vs Deferred vs Rejected?

27.14 Severity vs Priority — Refined

Theory: Severity = technical impact on functionality, set by QA, static (doesn't change unless functionality reassessed). Priority = business urgency to fix, set by PM/Lead, dynamic (changes with release dates, customers, revenue). Levels: Severity Critical/Major/Minor/Trivial; Priority High/Medium/Low. Four combos are classic interview traps: High/High (checkout fails — fix now), High/Low (crash on rare IE8 report page — important but deferrable), Low/High (homepage logo misspelled "Gogle" — cosmetic but brand-visible, fix now), Low/Low (settings-page color off — backlog). Enterprise Relevance: Severity/priority confusion causes wrong fixes first (P3 cosmetic prioritized over P1 data loss because a manager shouted). Deloitte answer: matrix + who sets what + independence + revenue/customer-visibility drivers.

Bug: "Gogle" on homepage → Severity: Low (cosmetic) | Priority: High (brand) → fix now.
Bug: rare-report crash → Severity: High | Priority: Low → schedule.
Rule: QA owns severity, business owns priority; neither overrides the other's axis.

High-Stakes Scenario: All bugs marked P1 by default ("everything urgent") — no prioritization possible, real P1s drown. Triage: severity/priority calibration workshop (anchor examples per level), default rules (data loss = Sev High minimum), weekly triage with business owner re-ranking. Anti-Patterns: Severity = priority conflation; everything P1; QA setting priority (business call); priority set once, never revisited; shouting-driven prioritization. Qs: Who sets severity vs priority? Give all four combo examples?

27.15 Root Cause Analysis — Refined

Theory: RCA finds systemic process cause, not symptom, to prevent recurrence — run within 48h for critical/escaped defects. Tools: 5 Whys (Toyota/Toyoda: ask Why iteratively 3-7 times until a fix would stop recurrence), Fishbone/Ishikawa (People, Methods, Tools, Environment, Data, Process), Fault-Tree for safety-critical audit. Example flaky suite: shared DB + missing async waits + 2GB CI runners (three contributing causes, one systemic gap: no automation review gate). Rule: blame process, not person; document actions; verify recurrence drops. Enterprise Relevance: Without RCA, teams fix symptoms (rerun green) while causes persist (flake returns weekly). Deloitte postmortems require: timeline, 5-whys, contributing factors, action items with owners + dates, recurrence metric.

Fail → Why no wait? → Why no review caught it? → Why no standard? Root: no automation review gate.
Actions: gate added (owner: SDET lead, due: Fri) + existing suite audited (owner: QA, due: next sprint)
Verify: flake rate 12% → 3% in 4 weeks, else reopen RCA.

High-Stakes Scenario: Prod outage postmortem blames "human error" (engineer typo) — same outage recurs (no process change). Triage: 5-whys past the person (why could a typo reach prod? no validation? no review? no canary?), fix the system (validation + review + canary), track recurrence. Anti-Patterns: Blaming people; single-cause thinking (usually 3+ factors); action items without owners/dates; RCA filed, never verified; skipping RCA for "small" escapes (pattern source). Qs: 5 Whys vs Fishbone — when each? What makes an RCA actionable vs theater?

27.16 Quality Metrics — Refined

Theory: Quality Metrics split into Product (Defect Density = defects/KLOC, Escaped Defects, DRE), Process (MTTD/MTTR, reopen rate, flaky rate), Release (coverage, pass rate). Key formulas: DRE

(Defect Removal Efficiency) = pre-release defects / total defects × 100 (world-class >95%, poor <85% = late/over-budget delivery); Leakage = post-release / total × 100; Density pinpoints risky modules. Track pre vs post-release together; enforce coverage gates (e.g., 70%) not vanity 100%; use trends (not snapshots) for go/no-go. Enterprise Relevance: Metrics justify quality investment and release decisions. "DRE 93.7%, escape 0.31/K transactions, flake 1.2%" tells a release story; "testing went well" tells nothing.

```
DRE = (120 / (120 + 8)) × 100 = 93.7% (good, near world-class)
Density = 42 defects / 20 KLOC = 2.1/KLOC (module X highest → focus review there)
Go/no-go: DRE trend ↑ + escapes ↓ + flake <2% + coverage gate met = GO
```

High-Stakes Scenario: Release approved on 95% pass rate — hiding 12% flake (real pass 83%) + 0% coverage on new payment module. Triage: pass-excluding-flake metric, new-code coverage gate, escape-rate trend per release; no-go until thresholds met or risk waiver signed. Anti-Patterns: Pass-rate vanity (includes flakes); 100% coverage mandate (wasted effort, gaming); snapshot metrics (no trend); metrics without owners/actions; counting tests instead of risk coverage. Qs: DRE vs leakage vs density — what does each reveal? Which metrics gate a release?

27.17 Automation ROI — Refined

Theory: Automation ROI = (Benefits – Costs) / Costs × 100 over 6-12 months. Costs: licenses, framework setup, scripting, training, maintenance (~30% yearly — the line everyone omits). Benefits: manual-hours saved × cycles, faster regression (release cadence), avoided production defects (NIST cost anchor: 100x), avoided hires. Forrester TEI: ~4.5x over 3 years, payback ~13 months; small suites (50-150 tests) pay back in 18-24 months. Biggest error: omitting maintenance. Prioritize high-frequency, stable, critical-path regression for max return. Enterprise Relevance: ROI justifies automation budgets and answers "why not automate everything?" (because ROI is negative for one-shot/unstable tests). Every automation proposal needs payback math.

```
ROI = ((434k - 139k) / 139k) × 100 = 212% Yr1; Yr2+ → 703% as dev cost amortizes.
Breakdown: 560 hrs/yr saved (~$42K) + 3 escapes avoided (~$90K) vs $139K build + $42K/yr maint.
Payback: month 4 (cumulative savings cross costs).
```

High-Stakes Scenario: "Automate all 2000 manual cases" proposed (\$500k, 18 months) — half are one-shot/unstable (negative ROI). Triage: rank by frequency × stability × criticality; automate top 70% high-risk (positive ROI, 4mo payback); leave rest manual/exploratory; re-measure yearly. Anti-Patterns: Omitting maintenance (30%/yr); counting gross savings as ROI; automating unstable/one-shot (negative return); no payback timeline; ROI measured once, never revisited. Qs: What goes in costs vs benefits? How do you prove payback to a skeptical manager?

27.18 What Should NOT Be Automated — Refined

Theory: Do NOT automate: one-shot tests (run once, scripting costs more than manual), unstable-UI areas (selectors churn faster than maintenance), subjective UX/aesthetics (human judgment: "does this feel right?"), CAPTCHA/security anti-bots (designed to block automation), media correctness (video/audio quality needs senses), rapidly churning features (automate after stabilization), flaky variable-output tests where maintenance exceeds value. Manual/exploratory wins for usability, ad-hoc edge discovery, print/PDF visual checks. Enterprise Relevance: Saying "no" protects automation ROI

and suite trust. Every inappropriate automation becomes either a flake (erodes trust) or maintenance sink (erodes ROI). Senior SDETs defend the boundary with cost math.

```
Automate: login API + 500-row pricing data-driven + smoke critical journeys +  
perf/API checks (lowest layer: API over GUI).  
Don't: logo beauty, drag-drop editor nuances, toast UX feel, CAPTCHA, one-off  
migration checks, churning beta UI.  
Rule: repetitive + stable + deterministic + data-driven + regression/smoke →  
automate; else manual/exploratory.
```

High-Stakes Scenario: Team automates CAPTCHA (fails), visual "vibe" checks (subjective failures), beta UI (rewrites weekly) — 30% suite flake from inappropriate automation. Triage: audit suite against NOT criteria, convert to manual/exploratory charters or delete, gate new tests on stability + repeatability + determinism. Anti-Patterns: "Automate everything" mandate; automating to hit coverage numbers; visual pixel-diff without tolerance/design-system stability; CAPTCHA workarounds in prod tests (security violation). Qs: Give 3 things never to automate and why? How do you push back on "automate everything"?